Interview Tutor

Architecture and design review

A local Python harness for interview practice

Implemented system | 4 October 2026

System intent and scope

Turn a supplied page or topic into practice, feedback, and revision notes

Implemented

- One local user and one source/topic per session
- Level and optional company/tier preference
- One question at a time, then evidence-based marks
- Coding or theory notes and a downloadable PDF

Deferred / excluded

- Code execution and automatic repair
- Adaptive questions after each answer
- Saved history, accounts, and billing
- Autonomous browsing and deployment

Company context influences practice; marks are not a hiring-readiness prediction.

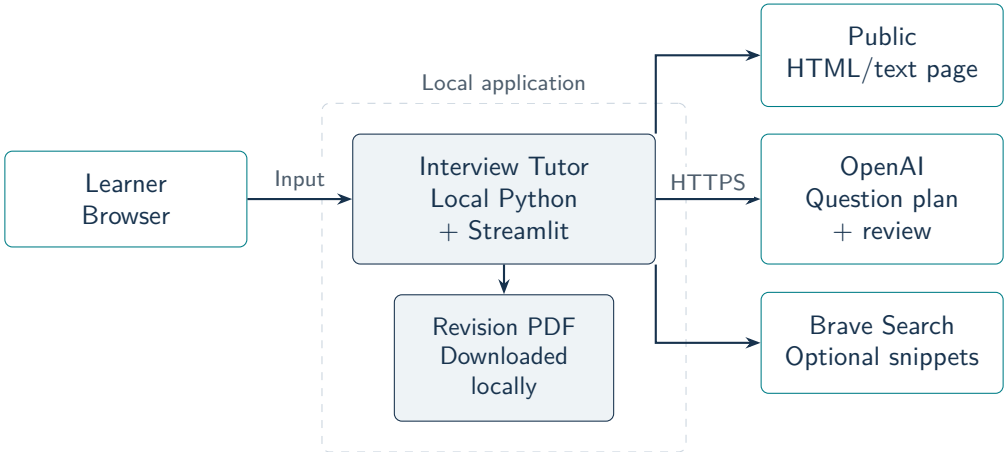
Source: `docs/requirements.md`; `README.md`

Requirements that drive the architecture

Requirement	Design response	Review evidence
SCOPE-008	Explicit Python orchestration	`TutorSession` owns the loop
FR-038	Fixed plan; bounded completion	No per-answer model calls; finalization once
FR-039	Restrict source retrieval	Public address checks, bounded text, paste fallback
FR-040	Optional fresh context	Model requests; harness executes one bounded search
FR-037	Evidence-based marks	Fixed rubric, exact learner quotes, local total
BUD-005/006	Bound and account for spending	Reservations, counters, known/unknown usage
FR-041	Actionable safe failures	Fixed error categories; retain usage before parsing

Source: `docs/requirements.md`; mapped checks in `tests/`

System context and runtime boundary



One application process; no database, worker queue, or execution sandbox.

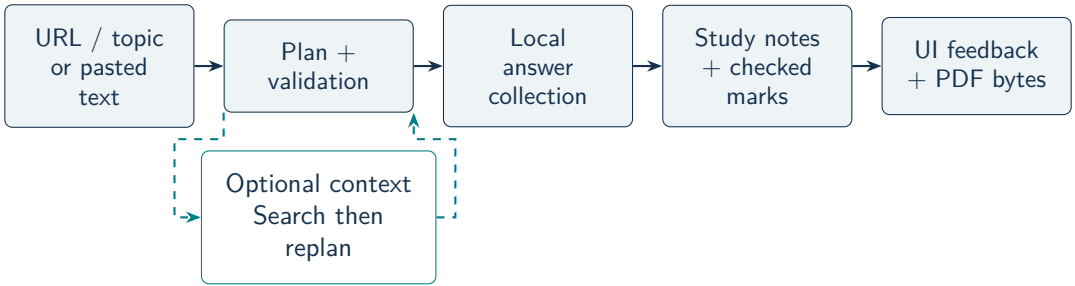
Source: `app.py`; `tutor/sources.py`; `tutor/openai_model.py`; `tutor/search.py`

Component responsibilities

Module	Owns
`app.py`	Input validation, Start, Streamlit session, cached PDF bytes
`tutor/harness.py`	Workflow, budgets, state, answers, review validation
`tutor/schemas.py`	Typed contracts, rubric, semantic validators, safe errors
`tutor/sources.py`	Topic/paste normalization and public-page retrieval
`tutor/openai_model.py`	Structured OpenAI responses and safe failure handling
`tutor/search.py`	Bounded Brave query and source provenance
`tutor/pdf_export.py`	Local document layout from existing notes
`config.py` / `secrets.py`	Limits and prices / provider-specific runtime keys

Source: Files under `tutor/`; `app.py`

End-to-end data flow



- Normally: one model plan and one final model review
- Research adds one search and one replanning call
- Each answer is saved locally; PDF rendering makes no model call

Source: `TutorSession.start/send/finish`; `app.py`; `build_pdf`

Source retrieval design

Normalize material before the harness asks the model

Selection rules

- Pasted text replaces URL fetching
- A topic can stand alone or focus a page
- One HTML/text page; no link traversal
- Retain an excerpt and label truncation

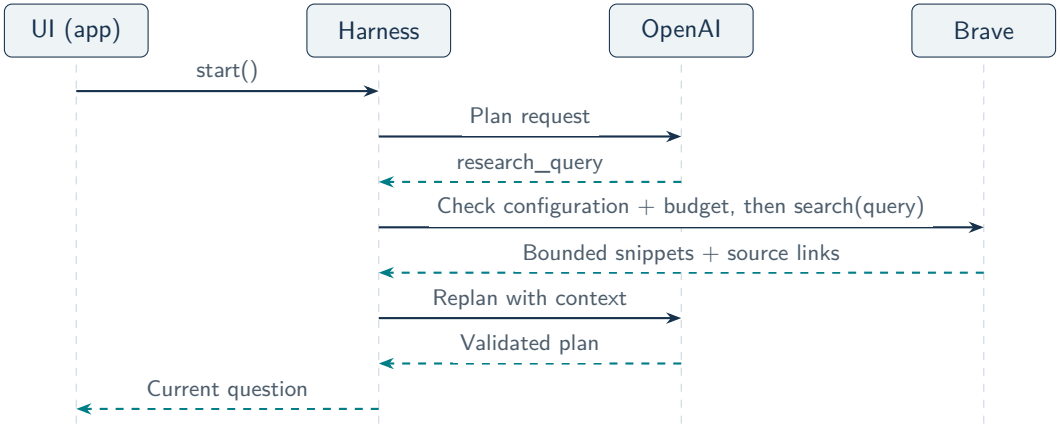
Network gates

- HTTP/S and standard ports only
- Reject credentials and private addresses
- Pin a checked IP; verify HTTPS hostname
- Recheck redirects and bound the response

Login, JavaScript, and unsupported pages need pasted material.
The socket/body timeout is not a strict overall deadline.

Source: `tutor/sources.py`; source integration checks in `tests/test_sources.py`

Conditional research sequence



Source: `tutor/harness.py`: `start`, `_research`; `tutor/search.py`

Session data model

Runtime state is a Python object, not a persistent run database

Record	Fields retained in memory
Source and settings	Material, title, URL, level, company context, configuration
`InterviewPlan`	Summary, categorized questions, optional research request
Learner interaction	Raw answers and the question/answer transcript
Research provenance	Query, bounded snippet context, source titles and URLs
`StudyNotes`	Concepts, approaches, feedback, follow-ups, nullable assessment
Usage and status	Attempt counts, reported tokens, estimated cost, uncertainty, notice
UI artifact cache	Generated PDF bytes in Streamlit session state

Source: `TutorSession.__init__`; `tutor/schemas.py`; `app.py`

Model response contracts

`InterviewPlan`

- Title and summary
- Distinct, nonblank categorized questions
- Plan mode: `research_query=null`
- Research mode: query + empty questions
- Cover every assessment category

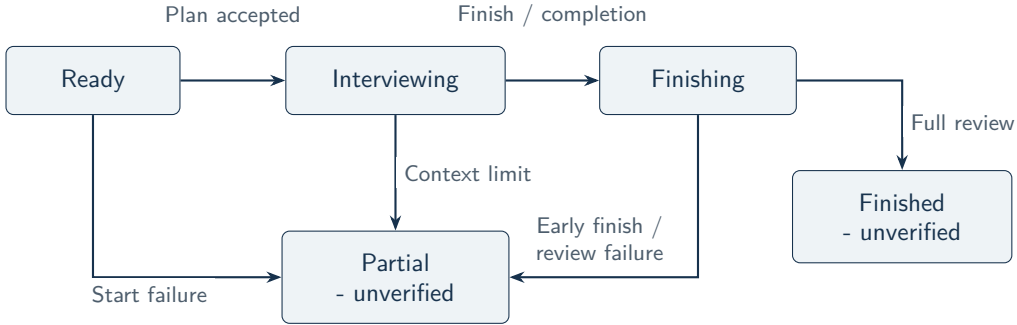
`StudyNotes`

- Summary, assumptions, key concepts
- Coding approaches only when relevant
- Feedback and common mistakes
- Further-practice questions and sketches
- Nullable evidence-based assessment

Strict JSON structure at the API boundary; semantic rules checked locally with Pydantic.

Source: `tutor/schemas.py`; `tutor/openai_model.py`: `_response_format`

Run state machine



Valid intermediate answers stay in `Interviewing`. Terminal sessions reject new answers. Planning/search happen inside `start()`; they are not separate stored states.

Source: `tutor/harness.py`: `start`, `send`, `finish`

Orchestration logic

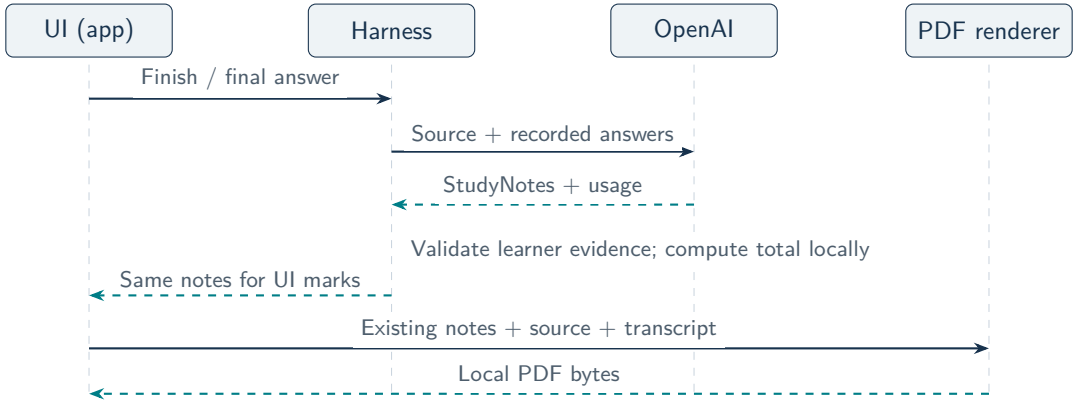
```
# Simplified pseudocode; guards remain owned by the harness.
plan = bounded_model_request(report=False)
if plan.requests_research:
    context = bounded_search(plan.query)
    plan = bounded_model_request(report=False, context=context)
    stop_partial_if_research_requested_again(plan)
accept_valid_plan(plan)

on_answer(text):
    validate_and_preserve_answer(text)   # no per-turn model call
    if report_context_over_limit: stop_partial()
    elif more_questions: show_current_question()
    else: finish_once()
```

The model proposes a tool request. Python authorizes, executes, validates, and stops.

Source: `tutor/harness.py`: `_request`, `_research`, `start`, `send`

Final review and export sequence



Source: `TutorSession.finish`; `InterviewAssessment`; `app.py`; `build_pdf`

Scoring and evidence validation

Criterion	Marks
Preparation demonstrated	3
Concept knowledge	3
Scenario application	2
Reasoning and clarity	2
Total computed in Python	10

- Integer marks must stay within each bound
- Every scored criterion needs an exact quote from one raw learner answer
- Missing evidence means `null`, not zero
- No total unless every criterion is assessed
- Invalid evidence discards the assessment

Provenance checks constrain the score; they do not prove that the model's judgment is correct.

Source: `tutor/schemas.py`: rubric, `validate_marks`, `validate_evidence`

PDF design and consistency

Render the same accepted notes object that drives the feedback screen

Report content

- Source, assumptions, and research links
- Scorecard with reasons and evidence
- Concepts, mistakes, and improvements
- Transcript and revision checklist
- Relevant follow-up questions

Presentation rules

- Coding approaches include code, dry runs, and complexity
- All code is labeled untested
- Missing final notes yield a partial report
- Escape text; wrap code for display
- Repeat table headers; number pages

Source: `tutor/pdf_export.py`; `app.py`; PDF checks in `tests/test_tutor.py`

Model adapter and failure containment

Inspect status and usage before attempting local JSON validation

1. Send `Responses.create` with a strict Pydantic-derived schema
2. Capture reported usage and inspect completion/refusal
3. Validate completed JSON and enforce local semantic rules
4. Return `ModelReply` or a fixed, sanitized `ModelRequestError`

Failure family	Session behavior
Authentication / quota / rate limit	Actionable safe notice; no automatic retry
Network / timeout / model access	Stop and retain existing material
Refusal / incomplete / invalid output	No fabricated review; retain known usage

Source: `tutor/openai_model.py`; `ModelRequestError`; `TutorSession._record_usage`

Budget and stopping design

Configured safeguards

- `gpt-4.1-mini`; estimated session cap $0.05
- 12 model-call attempts; one search attempt
- 40,000-byte prompt/schema allowance
- 1,800 plan / 3,500 report output tokens
- Bounded sources, context, and answers

Admission and accounting

- Reserve cost and final-review capacity first
- Reserve replan + report before search
- Count attempts before execution
- Replace reservations with known usage
- Unknown usage retains a conservative estimate

No automatic retries or model upgrades. Estimates are not an exact provider billing cap.

Source: Configuration snapshot: `tutor/config.py`; gates in `tutor/harness.py`

Trust boundaries and security limits

Boundary	Implemented control	Remaining limitation
Public page	Public-address checks, pinning, redirect/size limits	HTTP may be plain text; no strict overall deadline
Page/search text	JSON data and untrusted-context instructions	Prompt instructions are not a complete injection defense
Provider keys	Runtime lookup, separate key returned, Git ignore	Local file is not an encrypted secrets vault
Tool invocation	Fixed Brave endpoint and bounded query/results	Model decides relevance; snippets are unverified
Generated code	Never executed by the application	No correctness or sandbox verification
Local interface	Intended for one local user	No authentication or approved hosted deployment

Source: `sources.py`; `search.py`; `secrets.py`; `harness.py`; `.gitignore`

Verification strategy and evidence

Recorded baseline: 68 local tests passed on 4 October 2026

Test area	Evidence provided
State and contracts	Fixed planning, completion, malformed plans, idempotent finish
Boundaries and tools	Mocked public fetches, redirects, Brave bounds, key separation
Budgets and failures	Stop before spending; classify failures; preserve known usage
Assessment	Score bounds, single-answer quotes, source text cannot earn marks
UI and PDF	Start/input gates, configured research, matching marks and export
SDK transport	Real SDK with mocked HTTP; no sockets or real credentials

Live provider access, question relevance, semantic grading, and search quality remain unverified.

Source: `README.md`: verification record; all six modules under `tests/`

Architecture decisions and trade-offs

Decision	Reason	Consequence
Streamlit + one process	Small setup; Python throughout	UI reruns and workflow are coupled
Explicit harness	Visible guards and testable transitions	More orchestration code is maintained locally
Plan questions up front	Low call count and predictable cost	No answer-driven adaptive follow-ups
Pydantic + strict JSON	Shared typed response contracts	Semantic validators can still reject output
In-memory sessions	No database to operate	Restart/session loss can lose progress
Bounded snippet research	Fresh context through snippets	Limited context; relevance is model-dependent

Source: `README.md`; `docs/requirements.md`; implemented module boundaries

Operational design and known gaps

Current operation

- Local Streamlit process and browser session
- Synchronous source, model, and search calls
- Counters and safe notices shown in the UI
- Reusable PDF bytes after finalization
- Start uses configured model and budget

Gaps to review before wider use

- No durable history or recovery
- No in-flight cancel control or worker queue
- No persistent sanitized traces or timings
- No human-calibrated grading benchmark
- No authentication or deployment plan

Source: `app.py`; `TutorSession`; exclusions in `docs/requirements.md`

Review decisions for the next iteration

Extend the system only when a concrete practice need justifies it

1. **Question adaptation**
   Decide whether answer-driven follow-ups justify more calls and a revised budget.
2. **Grading quality**
   Compare feedback with human-scored examples before treating marks as calibrated.
3. **Session recovery**
   Add local persistence only if saving and resuming interviews is needed.
4. **Code verification**
   Require a separate sandbox/evaluation design before any execution or repair.

Source: Future options; current exclusions and approval boundaries in `docs/requirements.md`

Appendix: module interfaces

Interface	Contract
`load_source(url, topic, pasted_text, max_chars)`	Returns normalized `Source`
`TutorSession(..., model, config, search_client)`	Injects model/tool dependencies
`start() / send(text) / finish()`	Guarded state transitions
`generate(messages, report, max_output_tokens)`	Returns `ModelReply`; safe failure otherwise
`BraveSearch.search(query, max_results)`	Returns `SearchContext`
`validate_evidence(answers)`	Rejects unsupported assessment quotes
`build_pdf(..., notes, status, research_sources)`	Returns PDF bytes; no model invocation

Source: Public functions/classes in `tutor/`; signatures abbreviated for readability

Appendix: source map and supporting references

Repository evidence

- Scope and decisions: `docs/requirements.md`, `README.md`
- Runtime and orchestration: `app.py`, `tutor/harness.py`
- Contracts and adapter: `schemas.py`, `openai_model.py`
- Network and credentials: `sources.py`, `search.py`, `secrets.py`
- Artifact and limits: `pdf_export.py`, `config.py`
- Evidence: `tests/`, `docs/sample-notes.pdf`

Official API references for further reading

- OpenAI: Structured Outputs
- OpenAI: Error codes and Python error types
- Brave: Web Search API

Source: Repository snapshot: 4 October 2026. Configuration values describe this build, not a billing guarantee.